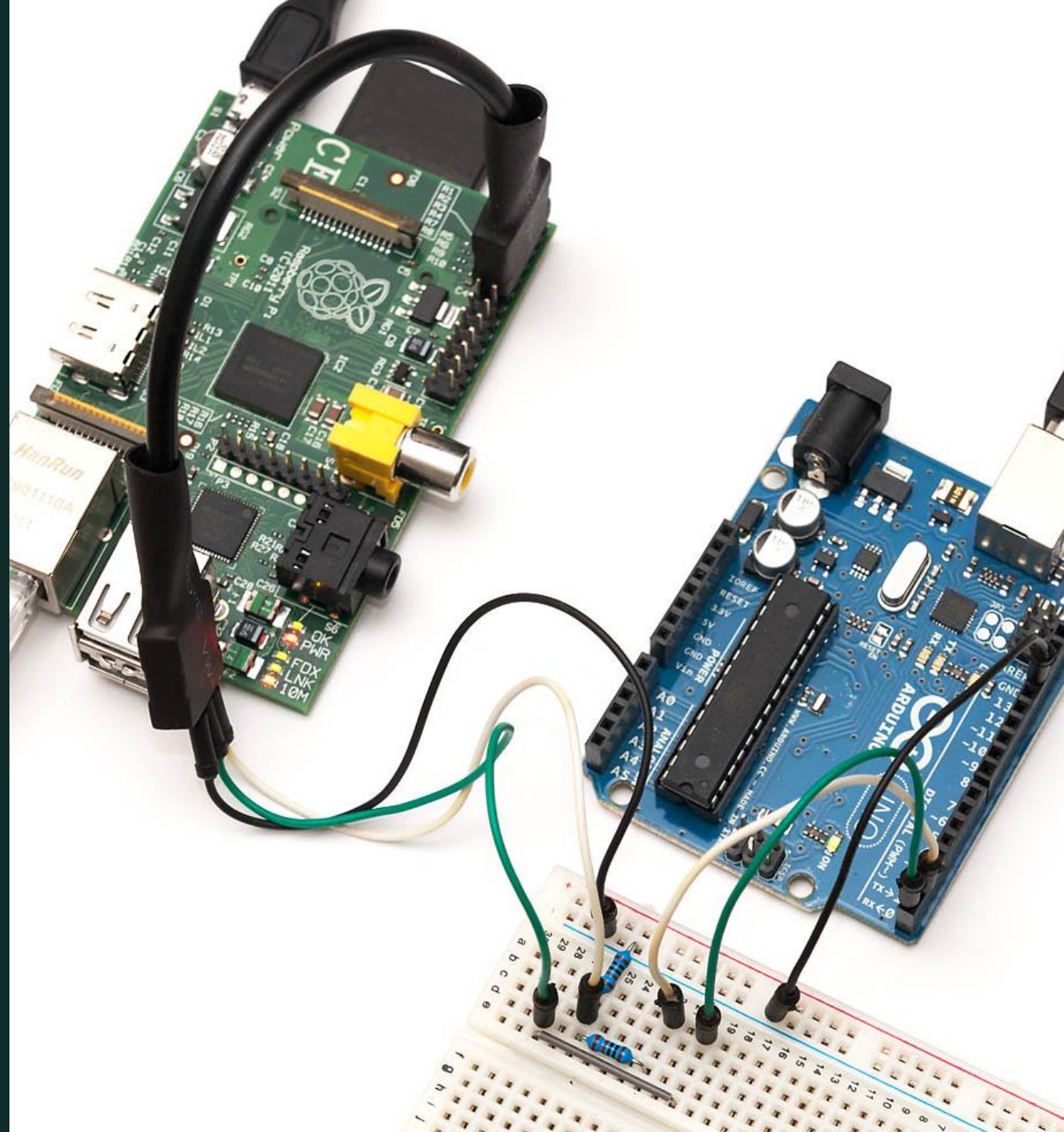


Introdução ao Raspberry Pi, Linux, Arduino e PlatformIO

Aula prática para começar do terminal ao sensor

físico + código + dados

Raspberry Pi como
computador



Mapa da aula

A ideia é entender o papel de cada peça antes de programar.

Ao final, o aluno deve conseguir: acessar um Raspberry Pi por SSH, navegar no Linux, compilar um projeto Arduino com PlatformIO e entender um fluxo IoT básico.



Raspberry Pi: computador pequeno para projetos físicos



Linux

Rede

GPIO

Python

Roda um sistema operacional completo em um cartão microSD.

Pode hospedar serviços: Node-RED, MQTT, InfluxDB, Grafana, scripts Python.

Usa General-Purpose Input/Output GPIO para interagir com sensores e atuadores.

É excelente para rede, armazenamento, dashboards e automação.

Ideia-chave: pense no Raspberry Pi como um mini servidor Linux conectado ao mundo físico.

Linux no Raspberry Pi: tudo começa no terminal

```
# Onde estou?  
pwd
```

```
# Listar arquivos  
ls -lh
```

```
# Entrar em uma pasta  
cd projeto
```

```
# Editar arquivo  
nano main.cpp
```

```
# Atualizar pacotes  
sudo apt update  
sudo apt upgrade
```

```
# Ver rede  
ip addr  
ping 8.8.8.8
```

```
# Serviços  
systemctl status ssh  
journalctl -u mosquitto
```

```
# Acesso remoto  
ssh pi@192.168.1.20
```

```
# Copiar arquivos  
scp firmware.cpp  
pi@IP:~/aula/
```

```
# Portas seriais  
ls /dev/ttyUSB*  
ls /dev/ttyACM*
```

O terminal permite reproduzir a aula em qualquer computador, mesmo sem interface gráfica.

GPIO: a interface entre Linux e eletrônica

Pinos comuns

3.3 V e GND

GPIO digital

I²C

SPI

UART

```
# ver o pinout no Raspberry Pi  
pinout
```

Cuidado elétrico

GPIO do Raspberry Pi é 3.3 V.

Não aplicar 5 V direto no pino de entrada.

Sempre usar GND comum.

Para UART com Arduino Uno, usar USB ou conversor de nível.

Primeiro teste

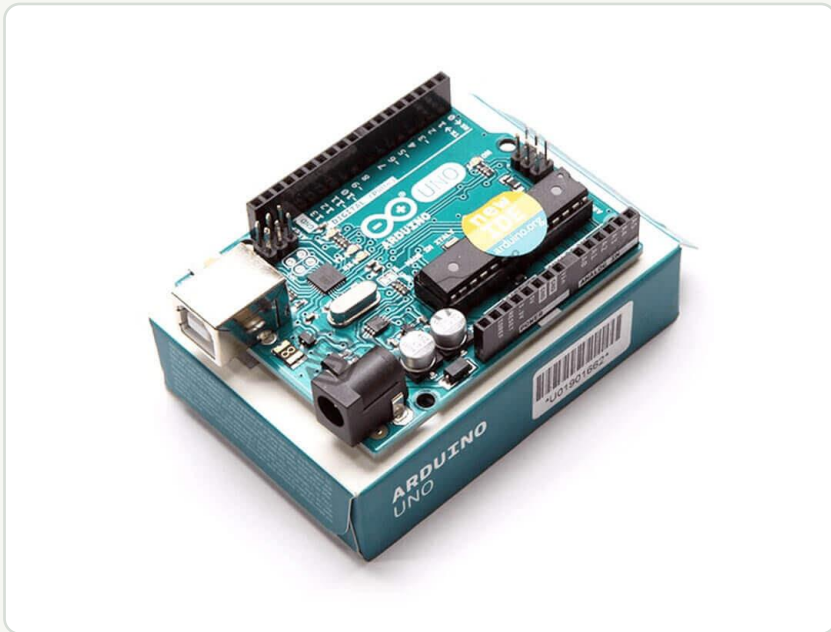
LED + resistor

Botão com pull-up/pull-down

Sensor I²C simples

Script Python com GPIO Zero

Arduino: microcontrolador para controle previsível



setup(): uma vez

loop(): repete

sem Linux

Estrutura mínima de um sketch

```
#include <Arduino.h>

void setup() {
  pinMode(LED_BUILTIN, OUTPUT);
}

void loop() {
  digitalWrite(LED_BUILTIN, HIGH);
  delay(1000);
  digitalWrite(LED_BUILTIN, LOW);
  delay(1000);
}
```

Arduino é ótimo para ler sensores simples, controlar atuadores e manter rotinas de tempo mais previsíveis do que em um sistema Linux multitarefa.

Raspberry Pi ou Arduino? Use os dois quando fizer sentido

Raspberry Pi

roda Linux

conecta à rede com facilidade

salva dados e roda dashboards

bom para scripts, banco de dados e web

Arduino

roda um programa dedicado

responde rápido a pinos e sensores

baixo consumo e baixo custo

bom para controle local e prototipagem

Regra prática: Raspberry Pi organiza, Arduino controla.

PlatformIO: projeto embarcado reproduzível

PlatformIO organiza placa, framework, bibliotecas e comandos em um projeto versionável.

```
aula-blink/  
├─ platformio.ini  
└─ src/  
    └─ main.cpp
```

```
[env:uno]  
platform = atmelavr  
board = uno  
framework = arduino  
monitor_speed = 9600
```

```
# compilar  
pio run  
  
# enviar para a placa  
pio run -t upload  
  
# monitor serial  
pio device monitor
```

Por que usar na aula?

mesmo fluxo em Windows, macOS, Linux e Raspberry Pi
facilita uso de várias placas
funciona via linha de comando e em VS Code

Prática 1: piscar o LED do Arduino Uno via PlatformIO

Passo a passo

conectar Arduino via USB

criar pasta do projeto

criar platformio.ini

criar src/main.cpp

compilar e enviar

abrir monitor serial

```
mkdir aula-blink && cd  
aula-blink  
pio project init --board  
uno
```

```
pio run  
pio run -t upload  
pio device monitor
```

Variação: trocar os tempos de HIGH e LOW para discutir frequência e duty cycle.

```
#include <Arduino.h>  
  
void setup(){  
    pinMode(LED_BUILTIN,  
    OUTPUT);  
}  
void loop(){  
    digitalWrite(LED_BUILTIN,  
    HIGH);  
    delay(2000);  
    digitalWrite(LED_BUILTIN,  
    LOW);  
    delay(1000);  
}
```

Checklist para a primeira aula prática

Antes da aula

imagem do Raspberry Pi gravada

SSH testado

PlatformIO instalado

cabos USB e LEDs

roteador ou rede definida

Durante a aula

todos digitam comandos

um erro por vez

registrar IP e porta

testar blink antes de sensor

salvar o projeto no Git

Resultado

Arduino piscando

Raspberry acessível

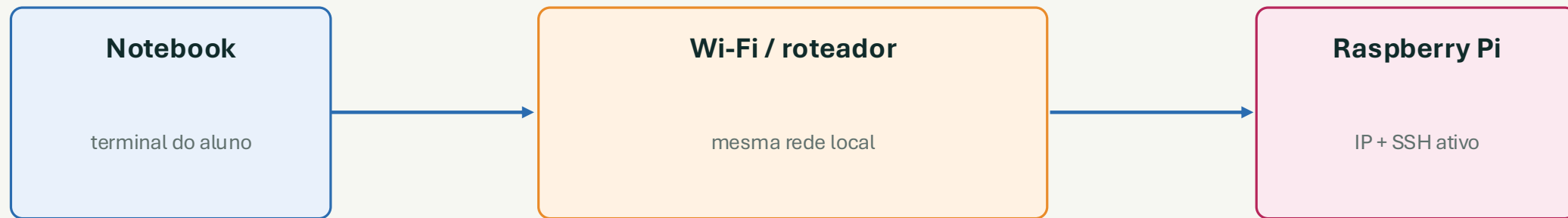
comandos anotados

um dado enviado

próximo sensor definido

Atividade no final da aula

Rede e SSH: trabalhar sem monitor



```
# descobrir IP no Raspberry Pi
hostname -I

# testar a rede
ping raspberrypi.local

# conectar
ssh pi@raspberrypi.local
# ou
ssh pi@192.168.88.227
```

Boas práticas

- anotar IP / nome do host
- usar cabo ou roteador dedicado em aula
- documentar comandos executados

Referências e recursos oficiais

Raspberry Pi Documentation — Raspberry Pi OS e GPIO

<https://www.raspberrypi.com/documentation/>

Arduino Documentation — hardware, software, IDE e CLI

<https://docs.arduino.cc/>

PlatformIO Documentation — Core CLI, Quick Start e VS Code IDE

<https://docs.platformio.org/>

Boas práticas de aula: sempre testar rede, portas USB e permissões antes da prática.